

Пользовательские типы данных

Структура - это пользовательский тип данных.
Данные, входящие в структуру, называют полями.

Структура - это набор из нескольких переменных, возможно различных типов, сгруппированных под одним именем для удобства обработки.

Пример структуры

Дата состоит из нескольких частей таких, как день, месяц, и год. Эти переменные можно объединить в одну структуру вида:

```
struct date { // объявление типа структуры
    int day;
    int month;
    int year;
};
```

```
struct date x; // объявление переменных этого типа
date y,z;      // объявление переменных этого типа
```

Создание структурных переменных

```
struct Student
{
    char Number[10], Fio[20];
    double S;
};
```

ИСПОЛЬЗОВАНИЕ СТРУКТУРНЫХ ПЕРЕМЕННЫХ

```
Student student, *p ;           //структурная переменная
                                //и указатель на структуру
Student S[100];                 //массив структур

Student *pstud;                 //указатель на структуру

Student* f(Student);
/*прототип функции с параметром структурного типа,
возвращающей указатель на объект структурного типа. */
```

инициализация структур

При декларации структурных переменных возможна их одновременная инициализация.

```
struct Student
{
    char Number[10], Fio[20];
    double S;
} student = {"107456", "Иванов И.И.", 6.53 };
```

или

```
Student student1 = {"107459", "Петров И.И." };
```

Если список инициализации короче, то оставшиеся поля заполняются «0»

Особенности создания структур

- поля не могут иметь атрибут, указывающий «класс памяти», данный атрибут можно определить только для всей структуры;
- идентификаторы полей могут совпадать с идентификаторами других объектов программы, т.к. шаблон структуры обладает собственным пространством имен;
- при наличии в программе функций пользователя шаблон структуры рекомендуется поместить глобально перед определениями всех функций и в этом случае он будет доступен всем функциям.

доступ к полям структуры

а) с использованием операции принадлежности (.) в виде:

имя_переменной_структуры.ID_поля

или

(*указатель_структуры).ID_поля

б) при помощи операции косвенной адресации (->) в виде:

указатель_структуры -> ID_поля

или

(&ID_структуры) -> ID_поля

Пример доступа к полям структуры

```
struct Student  
{    char Number[10], FIO[20];  
    double S;  
} student = {"107456", "Иванов И.И.", 6.53 }, *ps = &student;
```

```
student.S = 7.9;
```

```
(*ps).S = 8.1;
```

```
ps->S = 8.1;
```

```
(&student)->S = 7.9;
```

```
strcpy(student. FIO, "Иванов");
```

```
strcpy(ps-> FIO, "Сидоров");
```

Массивы структур

Структуры можно объединять в массивы

```
Student group[30];  
strcpy(group[10].FIO, "Иванов");  
strcpy(group[10].Number, "107435");  
group[10].S = 9.8;
```

Массивы структур

Работа с массивом структур через указатель, используя ->

```
Student group[30];  
Student * ps = group;  
strcpy(ps -> FIO, "Иванов");  
strcpy((ps+5) -> Number, "107435");  
(ps+10) -> S = 9.8;
```

Массивы структур

Работа с массивом структур через указатель, используя (*).

```
Student group[30];  
Student * ps = group;  
strcpy((*ps).FIO, "Иванов");  
strcpy(*(ps+5).Number, "107435");  
(*(ps+10)).S = 9.8;
```

Массивы структур

Работа с массивом структур через указатель, используя []

```
Student group[30];  
Student * ps = group;  
int i = 5; // № структуры в массиве  
strcpy(ps[i].FIO, "Иванов");  
strcpy(ps[i].Number, "107435");  
ps[i].S = 9.8;
```

Примеры программного кода

```
void main()
{
    SetConsoleCP(1251); // русский ввод/вывод в консоли
    SetConsoleOutputCP(1251);
    const int MAX = 5;
    struct Student // шаблон структуры
    {
        char fam[30]; // фамилия
        char name[20]; // имя
        int age; // возраст
        int mark[MAX]; // оценки
        double average; // ср. балл
    };
    Student group[30]; // массив данных по группе
}
```

```
int n = 0; //№ записи
while (true)
{
    cout << "Фамилия или 0: ";
    cin >> group[n].fam;
    if (group[n].fam[0] == '0')
        break;
    cout << "Имя: ";
    cin >> group[n].name;
    cout << "Возраст: ";
    cin >> group[n].age;
    cout << "Оценки: \n";
    double sum = 0;
    for (int j = 0; j < MAX; j++)
    {
        cout << j+1 << "): ";
        cin >> group[n].mark[j];
        sum += group[n].mark[j];
    }
    group[n].average = sum / MAX;
    n++;
}
```



```
for (int i = 0; i < n; i++)
{
    cout << group[i].fam << "\\t" << group[i].name << "\\t" <<
        group[i].age << "\\t";
    for (int j = 0; j < MAX; j++)
    {
        cout << group[i].mark[j] << "\\t";
    }
    cout << group[i].average;
    cout << endl;
}
```

Операции со структурами

Структуры можно присваивать:

`x = y;`

В этом случае оператор присваивания выполняет поверхностное копирование бит за битом полей переменной-источника и перенесение их в соответствующие поля переменной-цели (обе структуры-переменные имеют одинаковый тип).

С членами данных типа указатель могут возникнуть проблемы.

Структуры нельзя сравнивать:

```
if (x==y) cout << "*";
```

Копирование структур

`s2` в `s1` рекомендуется выполнять, используя:

```
memcpy(&s1, &s2, sizeof(тип_структуры));
```

Копирует `n` байтов, задаваемые третьим параметром из `s2` в `s1`.

Возвращает адрес `s1`.

Прототип функции описан в `string.h`

Вложенные структуры

Элементами структуры могут быть массивы и другие структуры. Допускается вложение структур.

Например, в структуре `person`, содержащей сведения - Ф.И.О., дата рождения, разрешается сделать дату рождения внутренней структурой `date` по отношению к структуре `person`.

Примеры кода

```
struct date
{
    int day, month, year;
};
struct Student//шаблон структуры
{
    char fam[30];//фамилия
    char name[20];//имя
    date birth;//дата рождения
    int mark[MAX];//оценки
    double average;//ср.балл
};
```



```
cout << "Фамилия или 0: ";
cin >> group[n].fam;
if (group[n].fam[0] == '0')
break;
cout << "Имя: ";
cin >> group[n].name;
cout << "Дата рождения \n день:";
cin >> group[n].birth.day;
cout << "месяц:";
cin >> group[n].birth.month;
cout << "год:";
cin >> group[n].birth.year;
```

```
for (int i = 0; i < n; i++)  
{  
    cout << group[i].fam << "\t" << group[i].name << "\t" <<  
    group[i].birth.day << "." << group[i].birth.month << "."  
<< group[i].birth.year  
<< "\t";  
}
```

Размещение структурных переменных в памяти

`sizeof(struct date)= 12`

`sizeof(struct person)=32` $20+4*3=32$

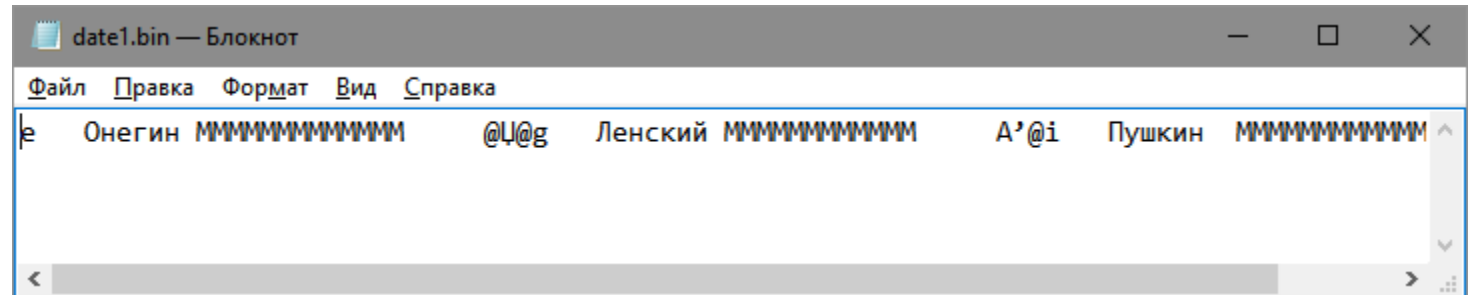
`sizeof(p)=160` $32*5=160$

Число байтов, выделенное под структурную переменную, не всегда равно **сумме длин ее элементов** из-за особенностей работы микропроцессора (выравнивание – **добавление к полям пустых байтов**). Иногда оказывается больше.

СТРУКТУРЫ И ФАЙЛЫ

Пример блочной записи в бинарный файл

```
cout << "работа с бинарным файлом" << endl;
file *f2;
fopen_s(&f2, "date1.bin", "wb");
struct emp {
    int code;
    char name[20];
    double salary;
};
emp s;
while (true)
{
    cout << "код (0-конец ввода): ";
    cin >> s.code;
    if (s.code == 0) break;
    cout << "фамилия: ";
    cin >> s.name;
    cout << "зарплата: ";
    cin >> s.salary;
    fwrite(&s, sizeof(s), 1, f2);
}
fclose(f2);
```



Пример блочного чтения из бинарного файла

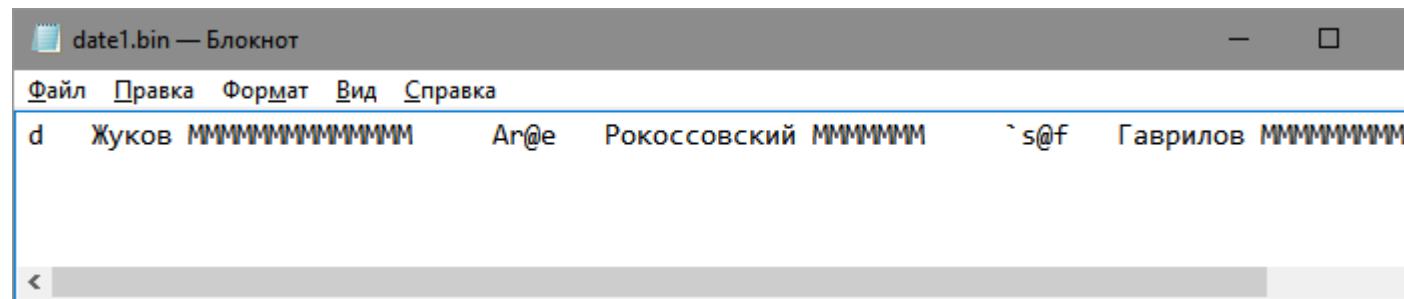
```
cout << "чтение данных" << endl;
fopen_s(&f2, "date1.bin", "rb");
while (fread(&s, sizeof(s), 1, f2))
{
    cout << "код : " << s.code << endl;
    cout << "фамилия: " << s.name << endl;
    cout << "зарплата: " << s.salary << endl;
}
fclose(f2);
```

Пример 2 блочной записи в бинарный файл

```
cout << "работа с бинарным файлом" <<
endl;
FILE *f2;
fopen_s(&f2, "date1.bin", "wb");
struct emp {
    int code;
    char name[20];
    double salary;
};
const int sizem = 20;
emp m[sizem];
for (int i = 0; i < sizem; i++)
    m[i].code = 0;
```

Пример 2 блочной записи в бинарный файл

```
int n = 0;
while (true)
{
    cout << "код (0-конец ввода): ";
    cin >> m[n].code;
    if (m[n].code == 0) break;
    cout << "фамилия: ";
    cin >> m[n].name;
    cout << "зарплата: ";
    cin >> m[n].salary;
    n++;
}
fwrite(m, sizeof(emp), n, f2);
fclose(f2);
```



Пример 2 блочного чтения из бинарного файла

```
cout << "чтение данных" << endl;
fopen_s(&f2, "date1.bin", "rb");
fread(m, sizeof(emp), 20, f2);
n = 0;
while(m[n].code != 0)
{
    cout << "код : " << m[n].code << endl;
    cout << "фамилия: " << m[n].name <<
        endl;
    cout << "зарплата: " << m[n].salary <<
        endl;
    n++;
}
fclose(f2);
```

Позиционирование в файле

- Каждый открытый файл имеет, так называемый *указатель* на текущую позицию в файле. Все операции над файлами (чтение и запись) работают с данными с этой позиции. При каждом выполнении функции чтения или записи, указатель смещается на количество прочитанных или записанных байт, то есть устанавливается сразу за прочитанным или записанным блоком данных в файле. Это так называемый *последовательный доступ к данным*.
- Последовательный доступ очень удобен, когда нам необходимо последовательно работать с данными в файле.

- Если необходимо читать или писать данные в произвольном порядке. Это достигается путем установки указателя на некоторую заданную позицию в файле функцией `fseek()`.

```
int fseek(file *stream, long offset, int whence);
```

```
int fseek(file *stream,  
          long offset, int whence);
```

offset – количество байт, на которое необходимо сместить указатель в направлении, указанном **whence**

Значения, которые может принимать параметр **whence**:

SEEK_SET 0 - *смещение выполняется от начала файла*

SEEK_CUR 1 - *смещение выполняется от текущей позиции указателя*

SEEK_END 2 - *смещение выполняется от конца файла*

Чтобы определить позицию указателя можно воспользоваться функцией `ftell()`.

```
long ftell(file *stream);
```

Возвращает значение указателя на текущую позицию файла.

В случае ошибки возвращает число (-1).

- **Дескриптор** - это целое число, которое система ставит в соответствие открытому файлу и в дальнейшем использует в операциях работы с файлом.

необходимо подключить к программному модулю заголовочный файл `<io.h>`.

```
int fileno (file *f); – определяет и возвращает значение дескриптора  
файла f;  
int fd=fileno (f);
```

```
long filelength (int fd); – возвращает длину файла, имеющего дескриптор  
fd , в байтах;  
long l= filelength(fd);
```

функции для работы с файлами

`void rewind(file *f);` – устанавливает индикатор позиции на начало файла

- `int fsetpos(FILE *stream, const long *pos)` -

Устанавливает значение указателя чтения-записи (указатель на текущую позицию) файла в позицию заданную значением по указателю `pos`. Возвращает нуль при корректном выполнении, и любое не нулевое значение при ошибке.

- `int fgetpos(FILE *stream, long *pos)` - Помещает в переменную, на которую указывает `pos`, значение указателя на текущую позицию в файле. Возвращает нуль при корректном выполнении, и любое не нулевое значение при ошибке.

- `long lseek(int handle, long offset, int fromwhere)` - функция устанавливает указатель записи-чтение файла на позицию `offset` в направлении, указанном `fromwhere` (по описанию `fseek`).
- `int chsize(int handle, long newsize)` - функция выполняет изменение размера файла. Признак конца файла устанавливается после байта с номером `newsize`. Возвращает 0, если размер файла был изменен, иначе - (-1)